

Jetpack Compose — Side Effects (Revision Note)

Why Side Effects Exist

A composable body runs on **every recomposition**. Anything that touches the outside world — network calls, toasts, listeners, analytics — must not sit directly in the body, or it will fire repeatedly and unpredictably.

```
kotlin

@Composable
fun Screen(userId: String) {
    val isValid = userId.isNotEmpty() // pure computation – fine in body
    fetchUser(userId)                 // WRONG – runs every recomposition
}
```

Quick Comparison

API	Runs when	Key	Cleanup	Typical use
LaunchedEffect	Enters composition + key changes	Yes	Auto-cancels coroutine	Fetch data on load or param change
DisposableEffect	Enters composition + key changes	Yes	Explicit onDispose { }	Listeners, players, timers, subscriptions
SideEffect	Every recomposition	No	None	Logging, analytics, syncing to non-Compose code
rememberCoroutineScope	Only when you call launch	No	Cancels when composable leaves	Button taps, form submit

1. LaunchedEffect

Runs a coroutine when the composable **enters composition** and again whenever the **key changes**. On key change the previous coroutine is **cancelled** before the new one starts.

```
kotlin
```

```

@Composable
fun UserProfile(userId: String) {
    var user by remember { mutableStateOf<User?>(null) }

    LaunchedEffect(userId) {                // re-runs only when userId changes
        user = fetchUser(userId)
    }

    Text(user?.name ?: "Loading...")
}

```

LaunchedEffect(Unit) vs LaunchedEffect(key)

- `LaunchedEffect(Unit)` — runs **once per composable instance**. `Unit` never changes, so recomposition never re-triggers it. Re-runs only if the composable leaves and re-enters composition.
- `LaunchedEffect(key)` — re-runs whenever the key value changes. Use when a parameter drives the work.

Cancellation is the superpower

```

kotlin

LaunchedEffect(searchQuery) {
    delay(300)                // debounce
    results = performSearch(searchQuery)
}

```

Each new keystroke changes the key, cancelling the pending `delay(300)`. Only the final query actually hits the network.

2. DisposableEffect

Same trigger rules as `LaunchedEffect`, but with a **mandatory** `onDispose { }` block for teardown.

`onDispose` runs in **both** cases:

1. Key changes → clean up the old resource, then set up the new one
2. Composable leaves composition → clean up

```

kotlin

```

```

@Composable
fun VideoPlayer(videoUrl: String) {
    val context = LocalContext.current

    DisposableEffect(videoUrl) {
        val player = ExoPlayer.Builder(context).build()
        player.setMediaItem(MediaItem.fromUri(videoUrl))
        player.prepare()
        player.play()

        onDispose {
            player.release() // always runs
        }
    }
}

```

Lifecycle for one player, many URLs:

```

Enter screen  → setup (play video 1)
URL changes  → onDispose (release) → setup (play video 2)
Leave screen  → onDispose (release)

```

Same shape applies to a music player, a `BroadcastReceiver`, a sensor listener, or a `LifecycleObserver`.

Registering a listener

```

kotlin

DisposableEffect(lifecycleOwner) {
    val observer = LifecycleEventObserver { _, event ->
        if (event == Lifecycle.Event.ON_RESUME) refresh()
    }
    lifecycleOwner.lifecycle.addObserver(observer)

    onDispose {
        lifecycleOwner.lifecycle.removeObserver(observer)
    }
}

```

3. SideEffect

Runs after **every** successful recomposition. No key, no coroutine, no cleanup.

```

kotlin

```

```

@Composable
fun UserProfileScreen(userId: String, userName: String) {
    SideEffect {
        analytics.log("profile_shown", "userId=$userId, name=$userName")
    }

    Text("User: $userName")
}

```

Why not just put the log in the body?

Technically the body statement would also run every recomposition. Two reasons to use `SideEffect`:

1. **Intent** — body code reads as pure UI computation; `SideEffect { }` explicitly says *"this touches the outside world, and yes, I want it on every pass."*
2. **Correctness** — `SideEffect` runs only after the recomposition **succeeds**. A body statement can run during a recomposition that Compose later discards.

Limits

Never put network or DB calls here — they'd fire on every unrelated state change:

```

kotlin

SideEffect {
    fetchUserFromApi(userId)    // WRONG - use LaunchedEffect(userId)
}

```

Use only for: logging, analytics, publishing state to non-Compose objects (system UI flags, third-party SDKs).

4. rememberCoroutineScope

Gives you a `CoroutineScope` tied to the composable's lifetime. You call `launch` **manually**, from an event handler.

```

kotlin

```

```

@Composable
fun LoginScreen() {
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var isLoading by remember { mutableStateOf(false) }

    val scope = rememberCoroutineScope()

    Column {
        TextField(value = email, onValueChange = { email = it })
        TextField(value = password, onValueChange = { password = it })

        Button(onClick = {
            scope.launch {
                isLoading = true
                try {
                    loginApi(email, password)
                } finally {
                    isLoading = false
                }
            }
        }) {
            Text(if (isLoading) "Please wait..." else "Login")
        }
    }
}

```

The scope is cancelled automatically when the composable leaves composition, so in-flight work doesn't leak.

The rule

Trigger

API

Automatic — screen enters, parameter changes

`LaunchedEffect`

Manual — user taps, swipes, submits

`rememberCoroutineScope`

Typical app flow:

- Login screen → user taps Login → `scope.launch { loginApi() }`
- Home screen → screen enters → `LaunchedEffect(Unit) { fetchUserDetails() }`

Caution

`rememberCoroutineScope` is tied to the **composable**, not the ViewModel — it dies on rotation. For work that must survive configuration changes, launch in `viewModelScope` and

call a ViewModel function from `onClick`:

```
kotlin

Button(onClick = { viewModel.login(email, password) })
```

5. snapshotFlow — Not a side effect, a tool used inside one

`snapshotFlow` converts Compose state into a cold `Flow` so you can apply Flow operators to it. It must be collected inside a coroutine — normally `LaunchedEffect`.

Debounced search

```
kotlin

LaunchedEffect(Unit) {
    snapshotFlow { searchQuery }
        .debounce(300)
        .distinctUntilChanged()
        .collect { performSearch(it) }
}
```

Compare with the naive version, which re-runs the whole effect on every keystroke:

```
kotlin

LaunchedEffect(searchQuery) {
    delay(300)
    performSearch(searchQuery)
}
```

Both work for debounce; `snapshotFlow` wins when you need real operators (`combine`, `filter`, `map`, `sample`).

Pagination trigger

```
kotlin

val listState = rememberLazyListState()

LaunchedEffect(listState) {
    snapshotFlow { listState.layoutInfo.visibleItemsInfo.lastOrNull()?.index ?: 0 }
        .distinctUntilChanged()
        .filter { it >= movies.size - 5 }
        .collect { viewModel.loadNextPage() }
}
```

`distinctUntilChanged()` plus an `isAppending` guard in the ViewModel prevents duplicate page requests during fast scrolling.

Decision Flow

```
Need to do something outside pure UI rendering?
|
|─ Triggered by user action (tap/swipe/submit)?
|   └─ rememberCoroutineScope (better: call a ViewModel function)
|
|─ Needs explicit teardown (listener / player / timer)?
|   └─ DisposableEffect(key) { ... onDispose { ... } }
|
|─ Suspend work driven by state or screen entry?
|   └─ LaunchedEffect(key) { ... }
|       └─ Need Flow operators on Compose state? → snapshotFlow inside it
|
|─ Just logging / syncing to non-Compose code?
|   └─ SideEffect { ... }
```

Rapid-Fire Q&A

Q: Does `LaunchedEffect(Unit)` re-run on recomposition? No. `Unit` never changes. It re-runs only if the composable leaves and re-enters composition.

Q: Does `LaunchedEffect(Unit)` survive rotation? No — the Activity is recreated, so the composable re-enters and the effect fires again. This is why one-time API calls belong in the ViewModel's `init` or behind a state flag, not in `LaunchedEffect(Unit)`.

Q: What happens to the old coroutine when a `LaunchedEffect` key changes? It is cancelled before the new one launches.

Q: When does `onDispose` run? On key change (before the new setup) and when the composable leaves composition.

Q: Can I call a suspend function inside `SideEffect`? No — `SideEffect` is not a coroutine scope. Use `LaunchedEffect` or `rememberCoroutineScope`.

Q: Why is `snapshotFlow` needed if state already triggers recomposition? Recomposition redraws the UI. `snapshotFlow` lets you observe state changes as a stream and apply operators like `debounce`, `filter`, and `distinctUntilChanged` before doing side-effect work.

Q: Difference between `rememberCoroutineScope` and `viewModelScope`?

`rememberCoroutineScope` dies with the composable (including on rotation). `viewModelScope` survives configuration changes. Prefer `viewModelScope` for anything the user shouldn't lose on rotation.